A Safety-Aware, Systems-Based Approach to Teaching Software Testing

Natalia Silvis-Cividjian
Computer Science Department, Vrije Universiteit, Amsterdam, The Netherlands, n.silvis-cividjian@vu.nl

ABSTRACT

Although no one doubts that ubiquitous software should be tested thoroughly, a dedicated course on software testing (ST) does not commonly feature in most CS curricula. Even when this is the case, existing courses often fail to offer a real-world testing experience, essential for a successful software engineering career. In our opinion, the main underlying reasons are twofold. First, abstracting software from its socio-technical system context, creates the false impression that ST "equals" unit testing based on functional requirements. Second, practicing on small, anonymously pre-engineered artifacts, conveys a detached and truncated view of the whole testing process. This paper describes an original approach to teach ST that addresses these two limitations by adopting a *system* engineering paradigm, with an additional strong emphasis on *safety*. Targeting CS graduates, the course consists of two modules. In the core module, students build fundamental testing knowledge in a software engineering context, and put this knowledge to work in both roles of developer and tester. In the project module, students get hands-on experience in engineering, and in particular testing, of microcontroller-based, safety-critical systems. Software risk and test experts from industry are actively involved in both modules, for advising, guest lectures and project steering. Enthusiastic students' evaluations throughout years demonstrate that cultivating safety- and hardware-awareness, although unusual for CS programs at non-engineering universities, creates a unique testing experience, while revealing remarkable technical and psychological trade-offs. All this brings more realism, and contributes to a meaningful and efficient learning in ST classrooms.

CCS CONCEPTS

• Software and its engineering → Software verification and validation • Computer systems organization → Embedded systems

ITiCSE '18, July 2-4, 2018, Larnaca, Cyprus

ACM ISBN 978-1-4503-5707-4/18/07...$15.00
https://doi.org/10.1145/3197091.3197096

KEYWORDS

Software testing education, Non-engineering CS curriculum, Safety-critical cyber-physical systems, Real-world hardware, Academia-industry cooperation

ACM Reference format:

Natalia Silvis-Cividjian. 2018. A Safety-Aware, Systems-Based Approach to Teaching Software Testing. In Proceedings of 23rd Annual ACM Conference on Innovation and Technology in Computer Science Education (ITiCSE'18). ACM, New York, NY, USA, 6 pages. https://doi.org/10.1145/3197091.3197096

1 INTRODUCTION

Ubiquitous Computing, or Internet of Things (IoT) is a trend that promises to populate our society with billions of networked, software-intensive devices. However, along with this success come real dangers, when we rely on devices that are not properly tested. In this context, a solid knowledge of software testing (ST) becomes an essential skill for the future IoT professional. Unfortunately, at this moment there is a discrepancy between the high expectations of the software industry, and the poor to non-existent testing skills of the fresh CS graduates. This gap forces industry to invest substantial resources in ST training on-the-job of its new-hires [1]. We believe that educators can reduce this gap by training CS students better in the state-of-the-art of ST, while offering them a flavor of what to expect when they join a company.

The problem is that stand-alone courses on ST are indeed rare. Although basic testing concepts are often embedded throughout implementation-oriented courses, such as programming, software engineering, robotics, or human computer interaction, most CS programs tend to emphasize development at the expense of testing. Recent surveys and panel discussions show that unfortunately, CS students spend only 0 to 27 hours on testing software [2]. Nevertheless, pioneer efforts have been made to promote ST as a standalone discipline, resulting in a few solid textbooks that followed the classic, landmark Meyer's "The art of software testing" published in 1979 [3]. This shifted the main challenge of designing a dedicated ST course from finding a good textbook, to finding realistic and representative artifacts to practice on. By artifacts we don't mean only code, but also requirements specifications, that also need to be verified and validated, before serving as a test oracle. Finding adequate artifacts to mimic the months, or even years- long process of testing millions of LOC, based on thousands of pages of specifications, is obviously not an easy

task. A widely used, simple solution is to create, or re-use anonymous, short requirements and code snippets. Examples are the well-known triangle problem, the next date calculation problem, the library management system, etc. This approach always works perfectly for teaching unit testing, which verifies that a software component functions as intended. Unit testing compares the observable results to the expected ones, specified in specially designed test cases (Figure 1).

However, nowadays' software is hardly ever an isolated unit. Instead, it is part of a complex socio-technical system, interacting with hardware, digital data, human users, or other computer systems. Interestingly, modern accidents involving software-intensive systems do not necessarily trace back to software faults; in some cases, they do not trace back to a single root cause at all, but to a component-interaction type of fault [4]. One can conclude from this that even properly tested software that *functions* as expected cannot prevent a system from failing. This could indicate that there is something wrong with the way this software has been tested. Is this "proper" testing maybe not so proper after all? In our opinion, there are two problems related to the current way of teaching ST that suggests revisiting of existing ST academic programs.

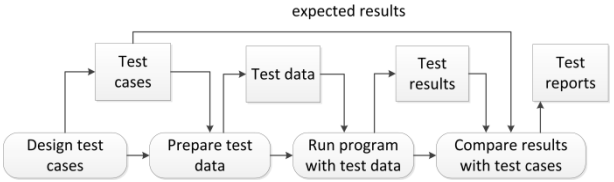

Figure 1: A global view of unit testing process. Adapted from [5].

First of all, *(1) Teaching only unit testing is not enough.* Traditional ST courses often come to an end before treating integration- and system testing. This is very unfortunate, because it deprives students of the opportunity to discover interesting problems, such as measuring unit inconsistencies between software modules, or hardware refusing to obey software commands. Second, also *(2) Teaching only to test for functionality is not enough.* Students often live under the false impression that testing can be stopped as soon as the product offers the intended functionality. However, this is only partly correct. Especially in mission-critical industry, a functional system is not yet ready to be put on the market. It needs a certification that assesses also other system-emergent quality attributes, such as security and safety. In real life, any safety-critical system must undergo a hazard analysis in the beginning of its engineering cycle. The results of this analysis are very valuable to software testers, as they generate system safety recommendations, later translated into software requirements, and eventually in safety-related test cases. Unfortunately, this interesting link between safety analysis and ST is not extensively exploited in traditional ST courses.

In this paper, we report on a successful solution to extend students' ST experience beyond unit testing of artificial, pure software components. The rationale was to reduce the gap between our graduates' ST skills and the actual industrial needs. Our challenge was to teach the fundamentals of ST in a short time, yet giving students a realistic idea of what they are likely to find when they join the workforce. The novelty of our solution lies in adopting a safety-aware, systemic perspective that embeds system engineering elements and physical devices. Designed for graduate CS students, the course consists of two components: (1) a core module, teaching basic black-box and white-box testing techniques, with a strong emphasis on safety, followed by (2) a full-time project, that consolidates students' testing knowledge by end-to-end engineering of a simplified, safety-critical embedded system. The core course has been given for ten years already, with minor changes. The systems project module was launched two years ago, as an elective module.

The remainder of the paper is organized as follows. We describe the core course outline in Section 2, followed by the systems testing project description in Section 3. Section 4 shares and discusses our teaching experiences and Section 5 discusses the related literature. Finally, we outline our conclusions and future plans in Section 6.

2 THE CORE COURSE

We started designing the core course in 2007, as a response to the stringent shortage of software testers perceived on the national job market. This was the time when, except for safety-critical segment, software testing in industry was often happening in an *ad hoc* fashion. The learning goal of this module was to teach students *how to check in a short time and with a reasonable coverage, that a software component functions as expected.* The course is an introduction to modern techniques to generate and execute test cases, for unit and integration testing levels. The main take-home message is that there is no single one silver-bullet technique, good for testing all software products. Therefore, we aimed to train students in recognizing which testing technique is more suitable in a specific case. This skill is, in our opinion, essential for a responsible test professional, always confronted with limited resources and business pressure.

The core course is two months long, offers a study load of 6 ECTS (168 hours) and is attended yearly by approximately 50 CS graduate students. Consisting of lectures and regular homework group assignments, the course focuses mainly on techniques for test generation from requirements (equivalence partitioning (EP), boundary value analysis, domain testing, combinatorial testing) and from code (control-flow, data-flow) [6-8]. A special attention is given to test evaluation with various adequacy criteria. Each year, we address a few special "hot" research topics, such as model-based testing, symbolic execution, testing from formal specifications, AI and testing, etc.

Until here, one may say that the course resembles any other traditional ST course. However, a few elements distinguish it from the traditional approach, as described next. For example, throughout the course, we encourage students to follow the news and report at the beginning of every lecture the so-called *bug of the day.* We also use an original way to psychologically

prepare the students for responsible testing, by making them aware of the implications of not-properly tested software. For their first assignment, students analyze in teams a notorious accident involving software-intensive systems, such as radiotherapy overexposure incidents, aircraft disasters, or space missions losses [9]. Various accident causality methods are used for this purpose, such as the traditional, reliability theory based Fault Tree analysis (FTA) and Failure Modes and Effects Analysis (FMEA), or the more modern systems-theory based, System Theoretic Accident Modeling and Process (STAMP) framework [4]. Another non-conventional element is the mini-project that immerses students in the testing scene, both as a developer and as a tester. First in the role of developers, students work in groups to specify, implement and test a Hangman game or a heart monitor interface. In this development phase, students practice unit and integration testing, with the freedom to decide which techniques and type of test process to use. Halfway through the project, the students swap their products and start the second, black-box testing phase. In this phase, students first set up a test plan, to test the product *and* its requirements specifications, with the freedom to use any testing technique. For example, a suitable test generation technique for the heart monitor user interface, would be EP with three classes: EC1-a valid class for normal blood pressure, EC2-an invalid class for low pressure, and EC3-an invalid class for high pressure. The expected output will be accordingly, Normal***, Alarm for low blood pressure****, or* ***Alarm for high blood pressure****.* This will generate however, "polite" test cases. We try to develop in our students a more creative and aggressive behavior to design test cases that aims to break the code. For example, why not try a test case with a NotANumber as value read from the sensor? Moreover, a heart monitor interface involves also safety issues that ask students to imagine hazardous situations, like the one when the value read from the sensor is for a long time equal to zero. Is this really due to a low blood pressure, or a defect sensor? Also, at the end of the course, we invite test experts from airlines, railways, and medical industries to present their best practices of organizing the testing process.

3. A SYSTEMS TESTING PROJECT

3.1 Preliminary Work

Following the Software engineering Curriculum Guidelines [10], recommending that "the curriculum should have a significant real-world basis", we decided to extend the core course with a hands-on project, where students use all their testing techniques arsenal on a *real* embedded system, in a fully immersive setting. The main learning goal of this project is to teach students *how to build and verify that a physical software-intensive system is functional and safe.* We aimed to extend students' testing experience up to system level, with the take-home message that software testing is just one-among-many phases in a system engineering process.

The result was a one-month long systems testing project (168 h) that accommodates the 24 most eager core course participants. The main challenge we faced when designing this module, was finding adequate systems-under-test (SysUT)s. First of all, (1) the physical systems should be tangible and easy to program for non-engineering students; (2) the systems should be safety-critical; (3) all systems should be simple, with a similar structure and underlying principle; (4) the systems should be equivalent in terms of engineering effort. Eventually, after brainstorming with industry experts, we proceeded with two systems, namely (1) a model railroad system, and (2) a water salinity & temperature control plant. Both are tangible, sensor-actuator systems, based on a simple close-loop, feedback control principle; both can be controlled with a microcontroller; both can host potentially critical hazardous situations, such as frontal and side train collision or derailments, or overheating and overflow in the water tank, respectively. Although they might look like toys, both systems successfully mimic modern, real-life safety-critical systems, namely a railway network management system, and an automatic insulin/glucagon pump. The big advantage is that our SysUTs, in contrast with the real ones, are open - students can touch and modify everything inside, and can access and write their own control software, free of public privacy and security issues. To speed up the development process, we built in advance the most part of the actuation and sensing physical infrastructure, while leaving some design choices open, to stimulate students' creativity. The main task students have to fulfill in this project is to specify, design, build and test a functional and safe control system. This control unit must be built around an NXP LPC1768 ARM microcontroller [11], supported by an open-source, on-online development platform including a C++ compiler. In order to strengthen its *embedded* attribute, the control unit is housed in multifunctional, transparent lid box. The box contains a breadboard, various knobs, switches, connectors, buttons, and an LCD screen to support interaction with the user (Figure 2). The wiring is configurable for both SysUTs.

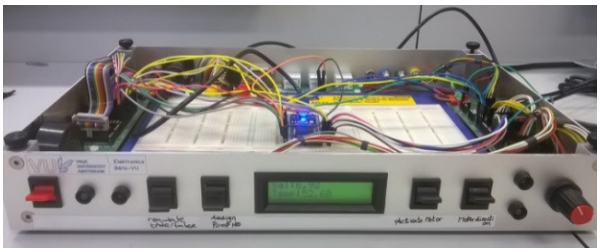

Figure 2: The embedded control unit used for the systems testing project.

3.2 Systems Under Test

Nowadays, with the imminent upcoming of autonomous, self-driving vehicles worldwide, timely and safely running of multiple trains is an actual and multidisciplinary problem, involving, more than ever, a strong computer science and engineering expertise. The first SySUT (Figure 3) is a H0 scale (1:87.1) miniature railway track that mimics a real-world railway control center. The actuators in this system are the DCC operated [12] motors of the locomotives and the turnouts (switches). The train's position on the track can be sensed with

14 Hall effect sensors mounted in the track. The goal is to build a control unit that runs two trains, and implements functionalities such as stopping at a platform, or activating a red light signal to protect a busy track section. Accidents such as collisions or derailments should be avoided at all times.

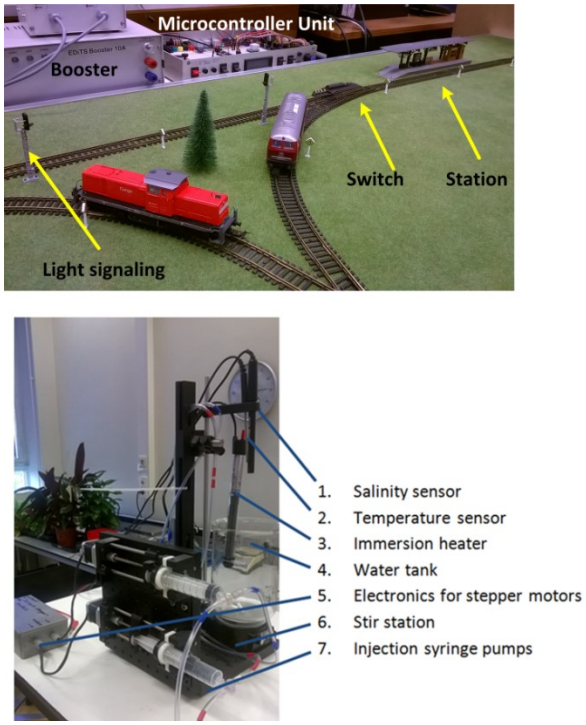

Figure 3: Systems under test. A model railway system (top). A water salinity- & temperature-control plant (bottom).

The second SysUT is inspired by another real-life problem, namely that of making the life of diabetic patients carefree and comfortable, by seamlessly keeping their blood glucose concentration (BGC) in a healthy range. The BGC is automatically regulated by injecting into the blood circuit a hormone, called insulin, when the measured level is too high, and injecting another hormone, glucagon, when this level is too low. An overdose of insulin is lethal. This is maybe one of the reasons why the real-life, fully automatic insulin/glucagon pump is still a technological dream. SysUT#2 mimics this safety-critical medical device by means of a water plant, as shown in Figure 3. The goal of the control unit for this system is in principle the same - to maintain the water salinity (instead of BGC) inside a specified range. Fluid injection is realized with two syringes, one for pure water (instead of insulin) and another one for salted water (instead of glucagon). The injection system is 3D printed, according to an affordable open-source design, suggested in [13]. In order to increase the complexity of the system, a second controlled variable was added, namely the temperature. The actuators in SysUT#2 are the syringe plungers' stepper motors and the immersion heater. Two sensors are used, to measure water salinity and temperature. Accidents such as underflow, overflow or overheating should be avoided at all times.

3.3 Project Organization

Each team of four students gets four weeks' time to engineer one of the two previously described control systems. They work in a lab, opened 7 hours a day. Students attend in Week 1 a few guest lectures from industry, introducing railway safety, diabetes care and testing embedded systems. Next, due to their non-engineering background, students need to follow a tutorial to get familiarized with the microcontroller basics, based on [14]. Next, an Agile development process starts, consisting of three weekly sprints. Each sprint is a sequence of the following system engineering activities: hazard analysis, (functional and non-functional) requirements specification, hardware wiring and software design, programming and testing. For each activity, students are free to use any technique they know or want to know better. They can experiment for example with formal specifications, or concentrate on system modeling, or on reliability testing. Both SysUTs are unique and thus shared, making planning the access to the test bed a crucial activity, as panic reaches a peak at the end of the project. Each week starts with a face-to-face progress meeting with a steering team, including the teacher, a TA and an IT safety consultant. At the end, the teams demonstrate their solutions in class, and present their findings and lessons learned. Guests from railway industry and diabetic care are invited to listen to the presentations. In their final document, students outline, argument and evaluate their testing strategy.

4 EVALUATION AND REFLECTION

Our course guides CS students in spending three months and between 150 and 300 hours on software testing, which is much more than reported elsewhere [2]. We evaluated the course using student surveys. The mini-project made students realize the importance of good requirements specifications. Students experienced how highly susceptible to interpretation is natural language, and how a shift to more formal requirement specification is highly needed. A discussion is currently taking place in the ST community, concerning whether the tester should face-to-face meet the developers or not. By observing our students' during the mini-project, we witnessed how confrontation between testers and developers unlocks a spectrum of often culturally determined complex emotions and behavior. Examples are brutality or in contrary, submissive hesitation in bringing up the bad, *(your code has a bug!)* type of news, the hurt ego of the self-confident programmer when confronted with an external bug report *(this is not a bug!)*, or the bug ownership dilemma *(this is not my bug!)*.

Guest lectures from industry were much appreciated by the students, because they could hear how much of what they learned is actually applied in practice, and in which context. Some students were disappointed, however, by the fact that testing organizations often make strategic choices, and do not use all the techniques we teach (preach) in class. Nevertheless, it was reassuring to hear from practitioners that testing is not at all a boring activity, but sooner an acrobatic exercise, that requires to continuously crafting smart trade-offs and compromises.

A project that brings hardware into play is an unusual component in a ST course, especially at a non-engineering university. Therefore, a detailed evaluation from both students' and educators' perspective is needed. The main question that needs to be answered is whether (1) *investing in an expensive (25.000 euros) physical infrastructure really creates a substantial positive effect on ST learning.* A less critical question is (2) *whether one month offers enough time to overcome the non-technical background of CS students and really get focus on testing.*

To start with, all students appreciated the daily opened lab, its collaborative, stimulating atmosphere, and the efficient and enthusiastic technical assistance offered by our systems electronics group. Students found that the project was realistic, and offered a flavor of what to expect when they step into the real testing world. We think that this is due to working in teams with tangible physical systems, and in close interaction with specialists from industry. Next, 80% of participants considered that they learned a lot and 13% were neutral. If we compare these results with the evaluations of only the core course (60% learned a lot, 28% neutral), we can conclude that the perceived learning effect in the project was a way higher. Let us try to summarize what students learned exactly. First, they learned (1) microcontroller programming skills interfacing software with real hardware. CS students had mixed feelings when working with this hardware. On the one hand, they were excited and eager to learn. On the other hand, they found it scary, because they discovered that electronic devices can be destroyed silently, without noise or smoke. And more surprising- this can be done by software! For example, by applying 5V instead of 3,3V to an analog input of a microcontroller, or by sending too long a command to a railway turnout that will first heat up, and later will melt down the solenoid. Moreover, students discovered that (2) testing of software embedded in real *systems,* is different from the *unit* testing theory taught in traditional ST courses, illustrated in Figure 1. Whereas in traditional courses the focus is exclusively on software, in our course, the focus has to be shared with adjacent hardware and other environment factors. For example, students struggled with the typical embedded systems testing problem of what *is* the expected output for a self-programmed thermometer? And after they managed to find an oracle, they discovered the next problem, that when hardware is in play, the measured output is never e*qual* to the expected output. The reasons are different errors, caused by inaccuracies in the measurements process, environmental noise, etc. In this context, students understood the role of calibration, noise filtering and the freedom a developer has to set its own non-functional requirements, such as system's accuracy. Another surprise was that a software command, even when sent properly, is not necessarily executed by hardware. This makes the observed behavior to deviate from the expected one. For example, locomotives do not always stop, turn-outs do not switch, syringe plungers do not advance. This happens because all physical components in the system can be defect; wires can be broken, fluid tubes can be obstructed, etc. All these are system hazards that need to be taken into consideration. We noticed that students had a tendency to overlook these risks in the beginning and identify only the most trivial hazards. This is in our opinion a naivety typical to software developers – assuming that the world is perfect, hardware already works, and humans respect the safety procedures. In real life, however, hardware fails, and humans idem - they forget, overlook, hurry and fail to follow procedures. The students understood that the challenge is to, regardless all this, build a safe system, with robust software that *knows* how to deal with faulty hardware- or human behavior.

This brings us to another point of learning, that of (3) testing other non-functional requirements, such as safety. Beside functional requirements, such as *"The train should stop at the platform*", students had to think about safety-related requirements, such as "*Frontal collision should be avoided even if a switch did not move correctly*". This safety constraint generates in fact a nice fault-injection type test scenario that simulates a fault in a rail switch, and checks that the software takes adequate decisions in order to prevent an imminent collision. In this way, students could feel how safety influences software testing. Moreover, safety influences the system design. For example, an alarm could be wired in the multifunctional box and programmed to detect an overheating in the water tank, for example. However, quality attributes such as functionality, safety and reliability, can sometimes contradict each other. For example, too many safety procedures will slow down the reaction time of the system, will frustrate the users and force them to be creative and overrule the procedures, or will overflow the microcontroller memory, etc. This required students to find a balance between safety and functionality. Students also learned (4) how to manage the project so that testing does not get compromised under time pressure. One month was in principle an appropriate project length. However, students discovered that freedom can be dangerous and panic grows if one does not plan properly. The NXP LPC1768 microcontroller and its mbed cloud C++ compiler turned out to be a good starting point - robust, with a steep learning curve, allowing enough time to focus on solution, rather than on implementation. However, because of their non-engineering background, CS students needed first a crash tutorial in electronics and encountered technical problems that caused frustration and unplanned delays. Successfully completing the project required therefore, a solid risk-based testing strategy, including priorities and choices, and the rationale behind them.

5 RELATED WORK

Existing dedicated ST courses come in different flavors. The majority aims to teach testing techniques as they are used in industry, while others adopt a more futuristic formal approach of model checking, or specialize in a certain type of testing (automatic, model-based, web- and mobile applications, etc). Others choose to focus mainly on the testing process. Our core course is focused mainly on standard and some off-the-beaten-path test design techniques. A similar "Bug of the day" idea to broaden student awareness of the cost of software flaws was reported in [15]. For the mini-project we used the ideas

suggested in [16]. Until recently, microcontrollers used to be very expensive, and their programming was an exclusive privilege of the computer engineers "guild". Fortunately, the situation has changed recently. Nowadays, affordable and steep learning curve microcontrollers such as Raspberry Pi, Arduino, NXP appeared on the market, allowing also non-technical CS students to successfully master embedded systems programming. This is demonstrated by an increasing number of reports on using microcontrollers in CS teaching [17-19]. Safety emphasis and exposure to physical devices for teaching software engineering have been recently reported in [20], with findings similar to ours. However, as far as we know, there are no similar attempts made for teaching particularly embedded software testing. This is mainly because, due to public safety or scalability issues, using real-world safety-critical systems in classrooms is, unfortunately, a bridge too far. A miniature, simplified physical model, is an excellent compromise in this case, as it brings students a way closer to a real setting than, for example, a computer simulation. A few attempts in this direction that inspired us in our design, used microcontrollers in combination with model trains [21, 22] and wet chemical reactors [23, 24], for teaching science and engineering.

6 CONCLUSIONS AND FUTURE WORK

We described a graduate course that teaches ST in a broader context of system engineering. Besides traditional learning, students spend up to 200 h on project-based testing of artifacts, varying from small size software modules, up to simple cyber-physical systems. The course creates a unique, engaging testing experience, due to: (1) a special emphasis on safety alongside functionality, (2) integrating testing with development, (3) working with real-world hardware, and (4) a close interaction with specialists from industry. We believe that this teaching approach can contribute to educating competent and responsible software testers, who get a flavor of what to expect when they join a company. Microcontroller-based miniature systems, such as railroad tracks and wet chemical plants, are an exciting environment for teaching safety-critical systems testing. In the future, we plan to extend the range of SysUTs towards the Internet of Things, such as autonomous cars networks. Also, we intend to extend testing to other non-functional system-emergent properties, such as security and reliability. Possibly, we will shift the microcontrollers programming tutorials to the undergraduate CS programme. We would also like to get a better insight into the testing teams' psychology.

ACKNOWLEDGMENTS

The author would like to thank Hans van Vliet for his idea to set up a dedicated ST course at the Vrije Universiteit. Jaap van Ekris, for his help to design and steer the systems testing project. Both VU Electronic Engineering and VU Mechanical Instrumentation groups, especially John de Roo and Niels Althuisius, for crafting wonderful systems under test. Wan Fokkink, for proofreading the manuscript, and giving the green light to build this (unusual for our CS department) hardware infrastructure. Vahid Garousi, James Hamblen, Neil Harrison, Garry Mabbott, Adithya Mathur and Tim Wilmshurst, for their help in fabricating challenging ST assignments. Finally, all the students and TAs, especially Sebastian Österlund, Petar Vukmirovic, Eline van Mantgem and Jacco van Splunteren, who contributed to making this course better.

REFERENCES

[1] R. Pham. et al. 2017. Onboarding inexperienced developers: struggles and perceptions regarding automated testing. Software Quality Journal, 25, 4, 1239-1268.

[2] T. Astigarraga et al. 2010. The Emerging Role of Software Testing in Curricula. In Proceedings of the IEEE Conference Transforming Engineering Education.1-26.

[3] G. J. Meyer.1979. The Art of Software Testing. John Wiley & Sons.

[4] N. G. Leveson. 2012. Engineering a Safer World: Systems Thinking Applied to Safety. MIT Press.

[5] I. Sommerville. 2004. Software Engineering. Addison Wesley.

[6] A. P. Mathur. 2008. Foundations of Software Testing. Addison-Wesley Professional

[7] L. Copeland.2004. A Practitioner's Guide to Software Test Design. Artec House Publishers.

[8] M. Pezze, M. Young. 2008. Software Testing and Analysis : Process, Principles and Techniques. John Wiley and Sons.

[9] P. A. McQuaid. 2012. Software disasters—understanding the past, to mprove the future. Journal of Software: Evolution and Process, 24, 5, 459-470.

[10] P. Bourque and R. E. Fairley. 2014. Guide to the Software Engineering Body of Knowledge. IEEE Computer Society.

[11] mbed NXP LPC1768 microcontroller. 2017. Retrieved from https://developer.mbed.org/platforms/mbed-LPC1768/.

[12] Digital Command Control (DCC). 2017. Retrieved from http://www.nmra.org/dcc-working-group.

[13] B. Wijnen et al. 2014. Open-Source Syringe Pump Library. PLoS ONE, 9, 9, e107216.

[14] R. Toulson and T. Wilmshurst. 2017. Fast and Effective Embedded Systems Design: Applying the ARM mbed. Elsevier.

[15] D. E. Krutz, and M. Lutz. 2013. Bug of the Day: Reinforcing the Importance of Testing. In Proceeding of Frontiers in Education Conference, 1795-1799.

[16] N. Harrison. 2010. Teaching Software Testing from Two Viewpoints. The Journal of Computing Sciences in Colleges. 26, 2, 55-62.

[17] E. A. Brand, W. L. Honig, and M. Wojtowicz. 2011. Intelligent Systems Development in a Non Engineering Curriculum. In Proceedings of ITiCSE. 48-52.

[18] S. Kurkovsky and C. Williams. 2017. Raspberry Pi as a Platform for the Internet of Things Projects: Experiences and Lessons. In Proceedings of ITiCSE, 64-69.

[19] J. O. Hamblen, and G. M. E. v. Bekkum. 2013. An Embedded Systems Laboratory to Support Rapid Prototyping of Robotics and the Internet of Things. IEEE Transactions on Education. 56, 1, 121-128.

[20] J. Cleland-Huang and M. Rahimi. 2017. A case study: Injecting safety-critical thinking into graduate software engineering projects. In Proceedings of ICSE. 67-76.

[21] J. Hamblen. 2017. Controlling a Model Railroad using mbed. Retrieved from https://os.mbed.com/users/4180_1/notebook/controlling-a-model-railroad-using-mbed/.

[22] L. Toms and D. West. 2017. Adding Fun to First-Year Computer Programming Classes with MATLAB, Arduino Microcontrollers, and Model Trains. Retrieved from https://nl.mathworks.com/company/newsletters/articles/adding-fun-to-first-year-computer-programming-classes-with-matlab-arduino-microcontrollers-and-model-trains.html.

[23] K. Crittenden, D. Hall and P. Brackin. 2010. Living With The Lab: Sustainable Lab Experiences For Freshman Engineering Students. In Proceedings of ASEE.

[24] G. A. Mabbott. 2014. Teaching Electronics and Laboratory Automation Using Microcontroller Boards. Journal of Chemical Education. 91, 9, 1458-1463.